PIC 10A 1A

TA: Bumsu Kim

This content is protected and may not be shared, uploaded, or distributed.

The author does not grant permission for these notes to be posted anywhere without prior consent.

UCLA

Today…

- HW5 Part 2 Hints
- 2-D and 1-D Vector Exercise: “Permutations”

HW5 Part 2 Hints

HW5 Part 2 Hints

- Two different approaches
- Strategy A: Use that $A = B$ if and only if $A \subseteq B$ and $B \subseteq A$
  - Pros: You can reuse my *HW3 Array solution* (on GitHub)
  - Cons: inefficient (=not fast) … but the efficiency doesn't matter for the HW score anyway

- Strategy B: Direct comparison after *sorting* elements
  - Pros: faster, and you can reuse the *array sort function* (on GitHub)
    - (Tuesday's version of the *array sort function* has some error, and it's fixed now)
  - Cons: Needs a special handling for duplicate elements
    - Approach 1: Ignore duplicates while getting user input
      - See *HW3 Array solution* for this!
    - Approach 2: Handle duplicates during the comparison, after sorting
      - See next slides for this

UCLA

Strategy A: Use $A = B$ *iff* $A \subseteq B$ and $B \subseteq A$

- First check if $A \subseteq B$:
  - For each member a of A:
    - check if a is in B
  - If every member of A is in B, conclude $A \subseteq B$.
- Secondly, change the roles of A and B and check if $B \subseteq A$

- For the membership check, use "has_member()" function in "Week6_HW3_Array_Sol.cpp"

```
/**
* Checks if the array [arr] has a member [n] or not
* @param arr: array to check
* @param sz: size of the array
* @param n: a data to check
* @return true if and only if arr has n
*/
bool has_member(int arr[], int sz, int n) {
	for (int i = 0; i < sz; ++i) {
		if (arr[i] == n) {
			return true;
		}
	}
	return false;
}
```

```
for ( /* each member a in A */) {
    /* check if B has a */
    // if so, do what?
    // otherwise, what?
}
```

UCLA

Strategy B: Direct Comparison After Sorting

- First sort both arrays, then compare the sorted arrays

- Handling duplicates:
  - Approach 1: Remove duplicates when getting input (See HW3_Array_Sol.cpp for this!)

A : 1, 4, 9, 16, 7, 11
B : 11, 7, 9, 16, 4, 1

$\xrightarrow{\text{Sort}}$

A : 1, 4, 7, 9, 11, 16
B : 1, 4, 7, 9, 11, 16

Easy to compare: Use part 1 of this HW!

  - Approach 2: Ignore duplicates after sorting the arrays

A : 1, 4, 9, 16, 9, 7, 4, 9, 11
B : 11, 11, 7, 9, 16, 4, 1

$\xrightarrow{\text{Sort}}$

A : 1, 4, 4, 7, 9, 9, 9, 11, 16
B : 1, 4, 7, 9, 11, 11, 16

Then compare numbers ignoring duplicates

UCLA

Comparison of Two Strategies

- Strategy A is pretty straightforward, but note that it is computationally inefficient
  - Let m = #(entries of A) and n = #(entries in B) counting duplicates.
  - We need m membership checks for A, and n checks for B
  - Each requires n or m comparisons in the worst case, so total $m * n$ comparisons

- On the other hand, the sorting method is much more efficient
  - 1. Sort A and B (requires $m \log m + n \log n$ computations – don't need to know why; covered in 10B)
  - 2. Comparison (requires at most $\min(m, n)$ comparisons)

UCLA

Summary

- To sum up, [Strategy B] – [Approach 1] is recommended, because you can reuse the following functions/code snippets that we *already* have:
  - read_inputs() [Week7_Array_Functions_sol.cpp]
    - with has_member() [Week6_hw3_array_sol.cpp] for a modification as in [Week6_hw3_array_sol.cpp] to ignore duplicates
  - sort() and its helper functions [Week7_Array_Functions_sol.cpp]
  - Comparison of two arrays [Part 1 of HW5]

UCLA

2-D and 1-D Vector Exercise: “Permutations”

Find all permutations of given numbers

Permutations

- Given a sequence of distinct numbers (as a vector<int>), find all permutations and return them as a vector of vectors

- For instance,

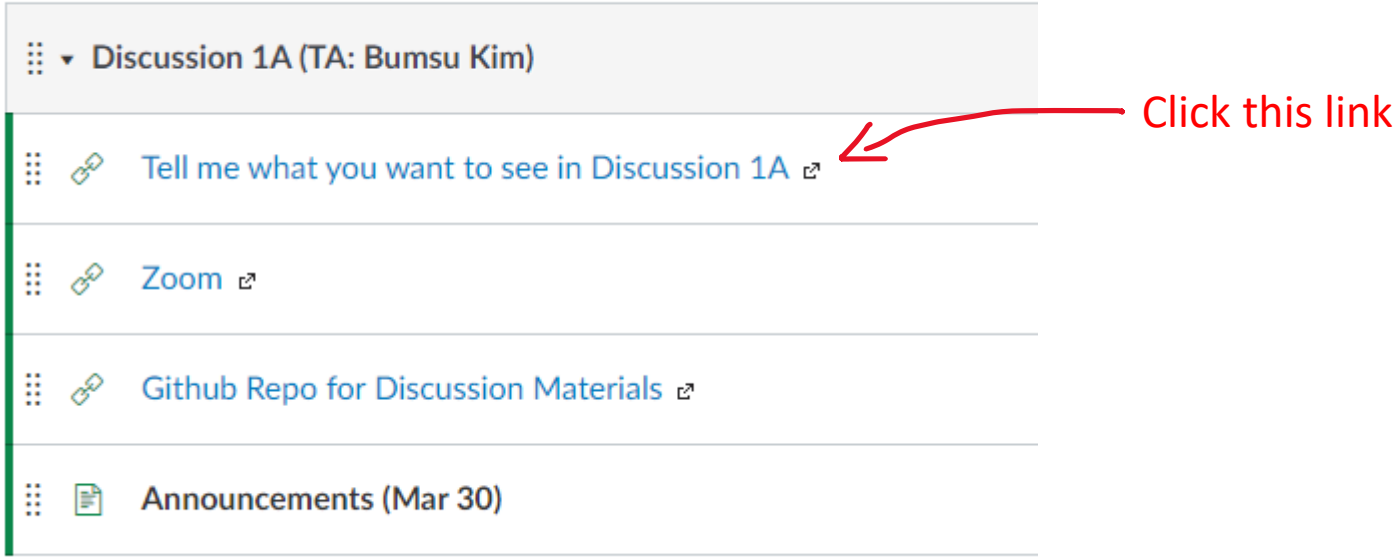
Given: {1, 2, 3}
as a 1-D vector

Answer:
{
{1, 2, 3}, {1, 3, 2},
{2, 1, 3}, {2, 3, 1},
{3, 1, 2}, {3, 2, 1},
}
as a 2-D vector
(vector of vectors)

- [Live Coding]

- Note: Solution is already on GitHub!

UCLA

Your Feedback is welcome

- Don’t hesitate to give a feedback on the discussion
- You can use a link in BruinLearn (Google Form)